Queue

- FIFO
- A linear data structure used to represent a linear list of elements. In queue, deletion of an element takes place only at one end, called ***Front*** and insertion takes place only at other end, called ***Rear***.
- ***Examples:***
  - Queue of people waiting to purchase tickets.
  - Queue of tasks waiting for printer, for access to disk storage.
- ***Applications:***
  - In a single program multiple requests can be kept in queue.

1

Operations on Queue

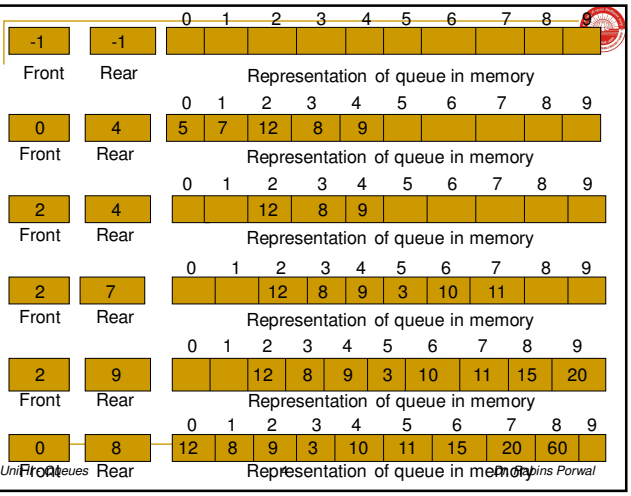
- **CreateQueue( )** - to create q as an empty queue.
- **Enque(*i*)** - to insert(add) element *i* in a queue q.
- **Dequeue( )** - to access and remove an element from queue q.
- **Peek( )** - to access the first element of the queue q without removing it.
- **isFull( )** - to check whether the queue q is full.
- **isEmpty( )** - to check whether the queue q is empty.

2

Queue as an ADT

- A queue of elements of any particular type is a finite sequence of elements of that type together with the following operations:
  - *Initialize* a queue to be empty.
  - Determine if queue is *empty* or not.
  - Determine if queue is *full* or not
  - If queue is not full, then insert a new element after the last element in a queue.
  - If queue is not empty, then retrieve the first element of a queue.
  - If queue is not empty, then delete the first element from a queue.

3



4

Limitations of a linear queue

- If the last position of the Queue is occupied, it is not possible to enqueue any more elements even though some positions are vacant towards the front position of the queue.
- This limitation can be overcome if we treat that the queue position with index 0 comes immediately after the last queue position with index MAX-1.The resulting queue is known as **Circular Queue**

5

Circular Queue

- This overcomes the problem of unutilized space in linear queue, implemented as an array. In the array implementation, there is a possibility that the queue is reported full even though slots of queue are empty.
- In short, just because we have reached the end of the array, the queue would not be reported as full. The queue would be reported full only when all the slots in the array are occupied.

6

Algorithm to insert an item in Queue

QINSERT(QUEUE, N, FRONT, REAR, ITEM)
This procedure inserts an element ITEM into a queue
1. If FRONT=1 and REAR=N, or If FRONT=REAR+1, then:
   Write: OVERFLOW and return
2. If FRONT = NULL, then:
   Set FRONT:=1 and REAR:=1
   else if REAR=N, then:
   Set REAR:=1
   else
   Set REAR:=REAR + 1
   [End of If structure]
3. Set QUEUE[REAR]:=ITEM
4.Return

7

Algorithm to delete an item from Queue

QDELETE(QUEUE, N, FRONT, REAR, ITEM)
This procedure deletes an element from a queue and assigns it to variable ITEM
1. If FRONT=NULL , then:
   Write: UNDERFLOW and return
2. Set ITEM:=QUEUE[FRONT]
3. If FRONT = REAR, then
   Set FRONT:=NULL and REAR:=NULL
   else if FRONT=N, then:
   Set FRONT:=1
   else:
   Set FRONT:=FRONT + 1
   [End of If structure]
4. Return

8

Applications of queues

- There are **several algorithms** that use queues to solve problems efficiently. For example, we use queue for performing **breadth-first search on a graph.**
- When the job are submitted to a networked printer, they are arranged in order of arrival. Thus, essentially, **jobs sent to a printer are placed on a queue.**
- Virtually **every real-life line (supposed to be) a queue.** For example, lines at ticket counter at cinema halls, railway stations, bus stand, etc, are queues, because the service, *i.e.*, ticket, is provided on first-come first-served basis.

9

Double Ended Queue (Deque)

- A linear list of elements in which elements can be added or removed at either end but not in the middle, *i.e.*, the elements can be added or deleted from both front or rear ends.
- There are two variations of a deque – an *input restricted deque* and *output restricted deque*, which are intermediate between deque & a regular queue.

10

Double Ended Queue (Contd…)

- *Input restricted deque:* A deque which allows insertions at only one end but allows deletions from both ends of the list.
- *Output restricted deque:* A deque which allows deletions from only one end but allows insertions at both ends of the list.
- Two possibilities that must be considered while inserting or deleting elements into the queue are:
  - When an attempt is made to insert an element into a deque which is already full, an *overflow* occurs.
  - When an attempt is made to delete an element from a deque which is empty, *underflow* occurs.

11

Priority Queue

- A collection of elements where each element is assigned a priority & the order in which elements are deleted & processed, which is determined by the following rules:
  - An element of higher priority is processed before any element of lower priority.
  - An element with the same priority are processed according to the order in which they are added to the queue.
- Example: A timesharing system – in which programs of high priority are processed first and programs with the same priority form a standard queue.

12

Array Implementation of Priority Queue

- In array implementation, insertion is easy but deletion of elements would be difficult because while inserting elements in priority queue they are not inserted in an order. As a result, deleting an element with the highest priority would require examining the entire array to search for such an element.

Priority Queue as one-way list

- A priority queue can be represented in memory by means of one-way list:
  - Each node in the list contains three items of information – an info field INFO, a priority no. PRNO and the link NEXT.
  - Node A will precede node B in the list if A has higher priority than B or if both the nodes have same priority but A was added to the list before B, *i.e.*, the order in one-way list corresponds to the order of priority queue.